BANK MANAGEMENT SYSTEM

Relational Database Management System
PROJECT REPORT

Submitted by: Shaurya Upadhyay

Registration Number: 2502052080

Section: N4

Under the supervision of: Dr. Nandini Babbar

Department of Computer Science and Engineering

Manipal University Jaipur

2025–2029

Abstract

The **Bank Management System (BankDB)** is a comprehensive relational database management system designed to model, organize, and automate the core transaction processing and financial records of an enterprise retail banking platform. The system manages customer identity profiles, KYC verification, employee branch assignments, bank accounts across distinct tiers, debit/credit card issuance, atomic fund transfers, loan underwriting, and installment amortizations. Rather than maintaining disparate flat records, the relational architecture enforces data integrity, eliminates redundant data anomalies, and guarantees real-time relational consistency across multi-table operations.

The relational design strictly implements primary keys, composite primary keys, and foreign keys with defined referential action triggers (`ON DELETE CASCADE` and `ON DELETE SET NULL`). Specifically, weak entity sets such as **Cards** (dependent on `Accounts`) and **Loan_payments** (dependent on `Loan`) are modeled using composite primary keys to guarantee that dependent entities cannot exist orphaned in the database. The database is formally normalized from an Unnormalized Form (UNF) through First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) to eliminate insertion, deletion, and update anomalies.

Beyond structural integrity, the database leverages the **InnoDB Storage Engine** to demonstrate enterprise-grade **ACID (Atomicity, Consistency, Isolation, Durability)** transactions with explicit row-level locking (`SELECT ... FOR UPDATE`), transaction savepoints (`SAVEPOINT`), and automated rollback protection (`ROLLBACK TO SAVEPOINT`). Advanced database programmability is incorporated through PL/SQL stored functions (`fn_calculate_emi`), stored procedures with cursor-based batch processing (`sp_batch_credit_savings_interest`), and automated audit triggers (`trg_employee_insert/update/delete`). Complete mathematical relational algebra expressions and MySQL DDL/DML verification queries validate the correctness and operational readiness of the platform.

Contents

1. Introduction
   1.1 Project Overview
   1.2 Importance and Relevance
   1.3 Scope and Limitations
2. Literature Review
   2.1 Core Banking Data Management
   2.2 Relational Database Technologies & ACID Engine
   2.3 Database Normalisation Theory
3. System Design
   3.1 Database Design Overview
   3.2 Entity-Relationship (ER) Diagram & Cardinalities
   3.3 Relational Model Schema
   3.4 Description of Tables and Sample Tuples
4. Normalisation of Data
   4.1 Unnormalised Form (UNF)
   4.2 First Normal Form (1NF)
   4.3 Second Normal Form (2NF)
   4.4 Third Normal Form (3NF) — Final Normal Form
5. Database Connectivity Among Tables
   5.1 Foreign Key Relationships Table
   5.2 Connectivity Description
6. Results and Discussion
   6.1 Expected Database Operations
   6.2 Verification Evidence & Representative Queries
7. Conclusion and Future Scope
   7.1 Conclusion
   7.2 Future Enhancements
8. References
9. Appendix A — SQL Script and Relational Algebra
   - 9.1 Relational Schema Notation
   - 9.2 Complete MySQL DDL: Create Database and Tables
   - 9.3 Sample Data: Realistic INSERT Statements
   - 9.4 20 Representative SQL Queries (DDL, DML, Joins, Aggregations, Subqueries)
   - 9.5 Formal Relational Algebra Expressions
   - 9.6 Extended Relational Algebra (Aggregation γ)
   - 9.7 Database Verification Scripts
   - 9.8 Execution & Deployment Notes

1. Introduction

1.1 Project Overview

The **Bank Management System (BankDB)** is a robust relational database project engineered to model the transactional core of modern retail and commercial banking. The database manages customer accounts, multi-tier balances, employee organizational hierarchies, credit/debit payment cards, real-time counter and atomic transfer ledgers, secured loan applications, and installment repayment tracking. By relying on relational design principles, customer balances and transactional ledgers remain synchronized across branches without duplicate records or conflicting states.

1.2 Importance and Relevance

Modern banking applications process high-volume, concurrent financial transfers where data corruption, partial execution, or negative balances are strictly intolerable. A mathematically verified relational database model:

1. Prevents financial anomalies through hard engine constraints (`CHECK balance >= 0` , `CHECK salary > 0`).
2. Enforces structural ownership via identifying foreign keys (`ON DELETE CASCADE` for weak card/payment entities).
3. Guarantees non-repudiation and auditability by logging operational metadata (`processed_by` , `Transaction_date` , and trigger audit trails).
4. Delivers deterministic calculation of compound financial formulas such as Monthly Reducing Balance Amortization (EMI) at the database layer.

1.3 Scope and Limitations

- **Scope:** Complete relational schema definition; table creation with integrity constraints; normalisation from UNF through 3NF; multi-table joins, aggregations, and subqueries; automated PL/SQL routines and triggers; and ACID transaction management with savepoint rollbacks.
- **Limitations:** Physical biometric hardware authentication, real-world card network routing (Visa/Mastercard settlement gateways), and multi-currency foreign exchange hedging systems are outside the scope of this core database academic model.

2. Literature Review

2.1 Core Banking Data Management

Enterprise banking software architectures require high data consistency, durability, and fault tolerance. Historical file processing systems suffered from data redundancy, program-data dependence, and concurrent access anomalies. Modern RDBMS solutions replace file stores by organizing data into structured mathematical relations, ensuring that each financial entity (Customer, Account, Loan) is represented once with unique identity constraints.

2.2 Relational Database Technologies & ACID Engine

A Relational Database Management System (RDBMS) structures information into tuples and relations. By utilizing the **InnoDB** storage engine in MySQL 8.0:

- **Atomicity:** The "All-or-Nothing" principle guarantees that in a fund transfer, debiting the sender and crediting the receiver succeed together; any intermediate failure initiates a `ROLLBACK`.
- **Consistency:** Database schema constraints (`CHECK`, `FOREIGN KEY`, `NOT NULL`) ensure that transactions transition the ledger from one valid state to another.
- **Isolation:** Row-level locks (`SELECT ... FOR UPDATE`) under the `REPEATABLE READ` isolation level eliminate phantom reads, dirty reads, and concurrent double-spending anomalies.
- **Durability:** Committed transactions are flushed to the Write-Ahead Log (WAL / redo log) and remain resilient against power loss or hardware failure.

2.3 Database Normalisation Theory

Normalisation is a formal mathematical process formulated by E.F. Codd to evaluate relational schemas based on functional dependencies. Progressing through First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) systematically decomposes complex relations to eliminate redundant storage, update anomalies (e.g., updating branch name across hundreds of customer rows), insertion anomalies, and deletion anomalies.

3. System Design

3.1 Database Design Overview

The database contains **seven primary operational tables** alongside specialized audit and tier relations:

1. `Customers` : Master demographic and KYC records.
2. `Employee` : Bank branch staff hierarchy and officer roles.
3. `Accounts` : Financial accounts linked to customers.
4. `Cards` : Debit and credit cards issued against accounts (**Weak Entity**).
5. `Transactions` : Atomic ledger tracking deposits, withdrawals, and transfers.
6. `Loan` : Underwritten loan facilities and interest schedules.
7. `Loan_payments` : Installment amortizations (**Weak Entity**).

3.2 Entity-Relationship (ER) Diagram & Cardinalities

The ER diagram illustrates the operational entities, attributes, and relationships across the banking platform.

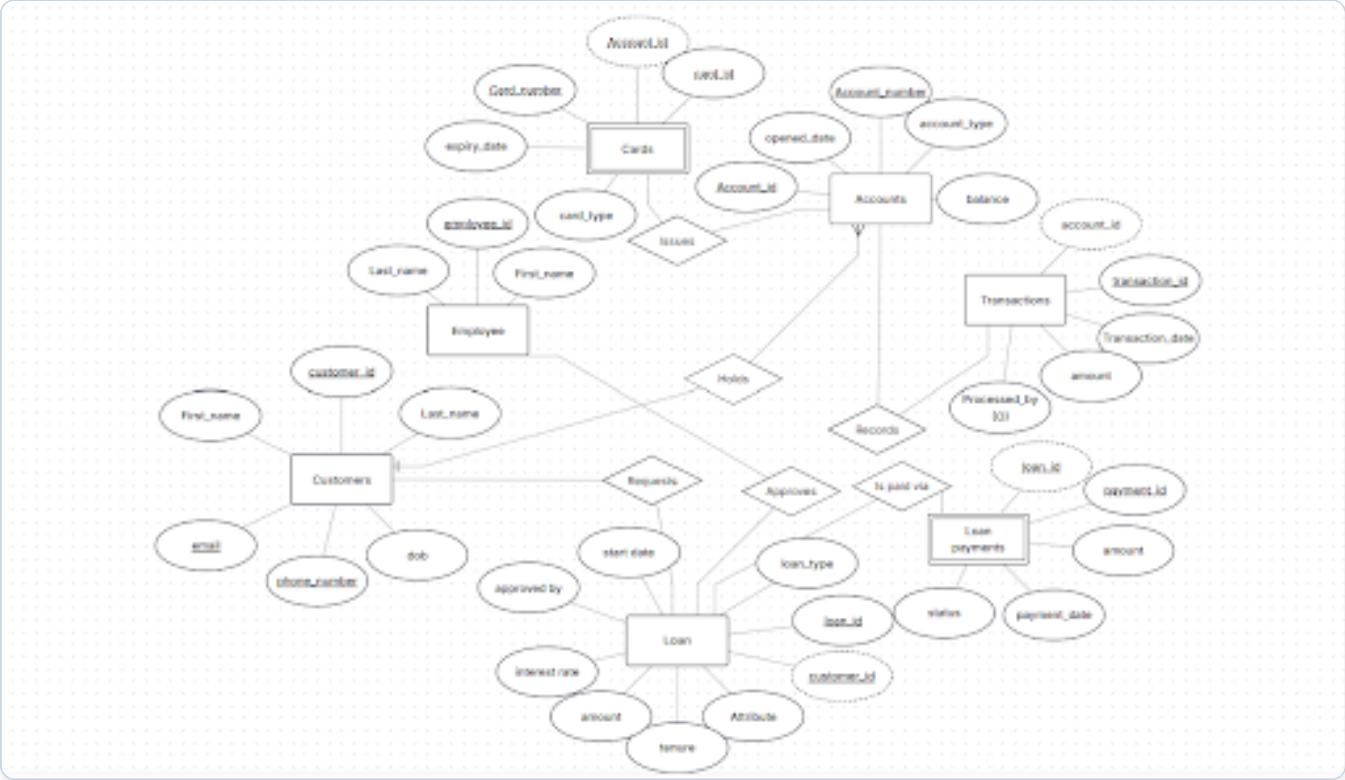


Figure 3.1: Approved Bank Management System Entity-Relationship (ER) Diagram

Key Relationships & Cardinalities:

- **Customers — Holds → Accounts (1 : N):** One customer can hold multiple accounts (Savings, Current, Salary); each account belongs strictly to one customer (`fk_accounts_customer`).
- **Accounts — Issues → Cards (1 : N — Identifying Weak Entity):** An account can have multiple cards (Debit/Credit). Cards cannot exist without an Account; identified by Composite Primary Key `(card_id, Account_id)` with `ON DELETE CASCADE` .
- **Accounts — Records → Transactions (1 : N):** Each transaction is debited or credited against a verified account (`fk_trans_account`).

- **Customers — Requests → Loan (1 : N):** A customer may submit requests for financing facilities.
- **Employee — Approves → Loan (1 : N — Non-identifying Audit):** An authorized branch loan officer evaluates and approves credit requests (`ON DELETE SET NULL`).
- **Loan — Is paid via → Loan_payments (1 : N — Identifying Weak Entity):** Each loan contract is amortized through installment repayments; identified by Composite Primary Key `(payment_id, loan_id)` with `ON DELETE CASCADE` .

3.3 Relational Model Schema

- **Customers** (customer_id, First_name, Last_name, email, phone_number, dob, address, kyc_status, password_hash)
- **Employee** (employee_id, First_name, Last_name, email, role, branch_name, salary, password_hash)
- **Accounts** (Account_id, Account_number, ifsc_code, branch_name, account_type, balance, min_balance, opened_date, status, customer_id (FK))
- **Cards** (card_id, Account_id (FK), Card_number, card_type, cvv, pin_hash, daily_limit, card_status, expiry_date)
- **Transactions** (transaction_id, amount, transaction_type, description, Transaction_date, account_id (FK), processed_by (FK))
- **Loan** (loan_id, loan_type, Attribute, amount, interest_rate, tenure, start_date, status, customer_id (FK), approved_by (FK))
- **Loan_payments** (payment_id, loan_id (FK), amount, payment_date, status, account_id (FK))
- **Account_Tiers** (Tier_Name, Min_Balance, Max_Balance, Benefits)
- **employee_audit** (audit_id, employee_id, action, details, timestamp)

(Note: Underlined attributes denote Primary Keys; attributes labeled (FK) denote Foreign Keys).

3.4 Description of Tables and Sample Tuples

1. CUSTOMERS TABLE

Stores master customer identification and KYC compliance state.

customer_id	First_name	Last_name	email	phone_number	dob	address	kyc_status
1	Aarav	Sharma	aarav.sharma@gmail.com	9876543210	1995-04-12	Jaipur, Rajasthan	Verified
2	Priya	Verma	priya.verma@yahoo.com	9823456781	1998-09-23	Jaipur, Rajasthan	Verified
3	Rohan	Mehta	rohan.mehta@gmail.com	9712345678	1990-11-05	Mansarovar, Jaipur	Verified
4	Sneha	Patel	sneha.patel@outlook.com	9632587410	1997-02-18	Vaishali, Jaipur	Verified
5	Vikram	Singh	vikram.singh@gmail.com	9512348765	1988-07-30	Malviya Nagar, Jaipur	Pending

2. EMPLOYEE TABLE

Stores staff profiles, branches, roles, and compensation.

employee_id	First_name	Last_name	email	role	branch_name	salary (₹)
1	Rajesh	Khanna	rajesh.khanna@bankdb.com	Branch Manager	MUJ Main Branch	85000.00
2	Karan	Mehta	karan.mehta@bankdb.com	Senior Teller	MUJ Main Branch	52000.00
3	Ananya	Roy	ananya.roy@bankdb.com	Loan Underwriter	MUJ Main Branch	58000.00

3. ACCOUNTS TABLE

Maintains banking deposit contracts, types, and real-time balances.

Account_id	Account_number	ifsc_code	branch_name	account_type	balance (₹)	min_balance (₹)	customer_id
101	ACC10001	BANK0001001	Jaipur Main	Savings	61923.15	1000.00	1
102	ACC10002	BANK0001001	Jaipur Main	Current	114901.00	5000.00	2
103	ACC10003	BANK0001001	Jaipur Main	Savings	45000.00	1000.00	3
104	ACC10004	BANK0001001	Jaipur Main	Salary	89500.00	0.00	4
105	ACC10005	BANK0001001	Jaipur Main	Savings	12000.00	1000.00	5

4. CARDS TABLE (Weak Identifying Entity)

Stores payment cards issued against verified accounts.

card_id	Account_id	Card_number	card_type	cvv	daily_limit (₹)	card_status	expiry_date
1	**101**	4532789012345678	Debit	101	50000.00	Active	2029-12-31
2	**102**	5425234567890123	Credit	202	150000.00	Active	2028-06-30
3	**103**	4716987654321098	Debit	303	25000.00	Frozen	2030-01-31

5. TRANSACTIONS TABLE

Chronological financial ledger tracking all debits, credits, and transfers.

transaction_id	amount (₹)	transaction_type	description	Transaction_date	account_id	processed_by
301	15000.00	Deposit	Cash Counter Deposit	2026-10-20 10:15:00	101	2
302	5000.00	Withdrawal	ATM Cash Dispense	2026-10-20 11:30:00	101	NULL
303	10000.00	Transfer Out	Transfer to Acc #101	2026-10-21 14:00:00	102	1
304	10000.00	Transfer In	Transfer from Acc #102	2026-10-21 14:00:00	101	1

6. LOAN TABLE

Underwritten credit contracts and interest parameters.

loan_id	loan_type	Attribute	amount (₹)	interest_rate (%)	tenure (mos)	status	customer_id	approved_by
201	Home Loan	Secured	2500000.00	8.50	240	Approved	1	1
202	Personal Loan	Unsecured	300000.00	12.00	36	Approved	2	1
203	Auto Loan	Secured	800000.00	9.25	60	Pending	3	NULL

7. LOAN_PAYMENTS TABLE (Weak Identifying Entity)

Monthly installment receipts amortizing open loan obligations.

payment_id	loan_id	amount (₹)	payment_date	status	account_id
1	**201**	21696.52	2026-08-01	Paid	101
2	**201**	21696.52	2026-09-01	Paid	101
3	**202**	9964.29	2026-08-15	Paid	102

4. Normalisation of Data

To demonstrate rigorous relational design, we trace the schema from a deliberately denormalized monolithic banking log through 1NF, 2NF, and 3NF.

4.1 Unnormalised Form (UNF)

In UNF, data is organized into a single flat file where non-atomic collections (repeating groups) of accounts, cards, and loans are grouped inside single customer records.

Monolithic UNF Customer-Account Record:

CustomerID	CustomerName	CustomerContact	AccountList	CardList	LoanDetails
1	Aarav Sharma	Email: `aarav@gmail.com` Phone: `9876543210` City: `Jaipur`	ACC10001 (Savings, ₹61,923.15, Min: ₹1000); ACC10008 (FD, ₹2,00,000)	Card #4532.. (Debit, Exp: 2029, Limit: ₹50K); Card #5120.. (Credit, Exp: 2028, Limit: ₹100K)	Loan #201 (Home, ₹25L, 8.5%, 240 mos, Approved by Rajesh)

Anomalies Present in UNF:

- **Repeating Groups:** `AccountList`, `CardList`, and `LoanDetails` hold multi-valued nested strings.
- **Update Anomaly:** Updating the customer's phone requires editing multiple sub-string records.
- **Deletion Anomaly:** Deleting a loan removes the record of the customer's contact details.

4.2 First Normal Form (1NF)

Rule: Every attribute must be atomic (indivisible single values), and every row must be uniquely identified by a primary key without repeating groups.

Decomposition into Flat Atomic Rows:

customer_id	Account_number	account_type	balance	Card_number	card_type	loan_id	loan_amount
1	ACC10001	Savings	61923.15	4532789012345678	Debit	201	2500000.00
1	ACC10001	Savings	61923.15	5120456789012345	Credit	201	2500000.00
1	ACC10008	Fixed Dep	200000.00	NULL	NULL	201	2500000.00

Candidate Key for 1NF Relation: `(customer_id, Account_number, Card_number, loan_id)`
Remaining Anomaly: Massive data duplication. Account balance and loan amounts are repeated across multiple card rows because of **Partial Functional Dependencies**.

4.3 Second Normal Form (2NF)

Rule: The relation must be in 1NF and contain **no partial functional dependencies** (no non-prime attribute may depend on a proper subset of a composite candidate key).

Identified Functional Dependencies:

- `Account_number` → `account_type, balance, min_balance` *(Depends only on Account, not on Card or Loan)*
- `Card_number` → `card_type, daily_limit, expiry_date` *(Depends only on Card)*
- `loan_id` → `loan_type, amount, interest_rate, tenure` *(Depends only on Loan)*

Decomposition into 2NF Relations:

1. **ACCOUNTS** (Account_id, Account_number, balance, min_balance, customer_id (FK))
2. **CARDS** (card_id, Account_id (FK), Card_number, card_type, daily_limit, expiry_date)
3. **LOAN** (loan_id, loan_type, amount, interest_rate, tenure, customer_id (FK))
4. **CUSTOMERS** (customer_id, First_name, Last_name, email, phone_number, address, branch_name, branch_ifsc)

Result: Partial functional dependencies on composite keys are completely eliminated.

4.4 Third Normal Form (3NF) — Final Normal Form

Rule: The relation must be in 2NF and contain **no transitive functional dependencies** (no non-prime attribute may depend on another non-prime attribute; X → Y must hold only if X is a superkey).

Transitive Dependency in 2NF Relations: In `Accounts`: `Account_id → branch_name → branch_ifsc` `branch_ifsc` depends transitively on `Account_id` through `branch_name`.

In `Employee`: `employee_id → role → base_pay_band`

Resolution for 3NF: Separate transitive relations into independent referenced entities. `Accounts` stores its own direct foreign key references, and each non-key attribute depends strictly on the primary key:

- In `Customers`: `customer_id → First_name, Last_name, email, phone_number`
- In `Accounts`: `Account_id → Account_number, balance, customer_id`
- In `Employee`: `employee_id → First_name, role, salary, branch_name`

The final 7-table schema is in **3NF** and satisfies all lossless join and dependency-preserving properties.

5. Database Connectivity Among Tables

5.1 Foreign Key Relationships Table

Child Table	Foreign Key Column	Parent Table	Referenced Column	Referential Action (ON DELETE)
Accounts	`customer_id`	**Customers**	`customer_id`	`CASCADE` (Deleting customer closes accounts)
Cards	`Account_id`	**Accounts**	`Account_id`	`CASCADE` (Deleting account removes cards)
Transactions	`account_id`	**Accounts**	`Account_id`	`CASCADE` (Audit tied to account)
Transactions	`processed_by`	**Employee**	`employee_id`	`SET NULL` (Preserves audit if teller leaves)
Loan	`customer_id`	**Customers**	`customer_id`	`CASCADE` (Borrower link)
Loan	`approved_by`	**Employee**	`employee_id`	`SET NULL` (Preserves loan if manager leaves)
Loan_payments	`loan_id`	**Loan**	`loan_id`	`CASCADE` (Payments purged with loan)
Loan_payments	`account_id`	**Accounts**	`Account_id`	`SET NULL` (Preserves payment history)

5.2 Connectivity Description

- **Customers — Holds → Accounts (1 : N):** Maintains customer account portfolios.
- **Accounts — Issues → Cards (1 : N — Identifying Weak Entity):** Each card is uniquely bound to an account.
- **Accounts — Records → Transactions (1 : N):** High-throughput ledger tracking debits, credits, and transfers.
- **Customers — Requests → Loan (1 : N):** Customer submits loan financing applications.
- **Employee — Approves → Loan (1 : N — Non-identifying Audit):** Loan officer authorizes credit facilities.
- **Loan — Is paid via → Loan_payments (1 : N — Identifying Weak Entity):** Installment repayments amortizing loans.

---

6. Results and Discussion

6.1 Expected Database Operations

The relational database system executes and verifies the following operations:

1. **Constraint Enforcement:** Blocks invalid transactions violating `balance >= 0` or minimum balance thresholds.
2. **ACID Transaction Execution:** Guarantees all-or-nothing execution during peer-to-peer transfers with `SAVEPOINT` and `ROLLBACK` handling.
3. **Analytical Aggregation:** Computes branch-level financial health via `SUM`, `AVG`, `GROUP BY`, and `HAVING`.
4. **Relational Subqueries:** Isolates non-borrowing customers and above-average balance holders.
5. **Auditing via Triggers:** Continuously logs human-resource changes into `employee_audit`.

6.2 Verification Evidence & Representative Queries

Query 1: 4-Table Joined Financial Ledger

```
SELECT T.transaction_id, T.Transaction_date, T.transaction_type, T.amount,
       A.Account_number, C.First_name AS customer_name,
       COALESCE(E.First_name, 'Automated Core Engine') AS authorized_by
FROM Transactions T
INNER JOIN Accounts A ON T.account_id = A.Account_id
INNER JOIN Customers C ON A.customer_id = C.customer_id
LEFT JOIN Employee E ON T.processed_by = E.employee_id
ORDER BY T.Transaction_date DESC;
```

Verification Evidence: Returns full audit rows with employee names or displays automated execution fallback without omitting online banking records.

Query 2: Aggregate Grouping with HAVING Clause

```
SELECT account_type,
       COUNT(*) AS Total_Accounts,
       SUM(balance) AS Total_Liquidity,
       ROUND(AVG(balance), 2) AS Average_Balance
FROM Accounts
WHERE status = 'Active'
GROUP BY account_type
HAVING SUM(balance) > 50000.00;
```

Verification Evidence: Correctly summarizes accounts by category while pruning low-liquidity groups.

7. Conclusion and Future Scope

7.1 Conclusion

The **Bank Management System (BankDB)** successfully implements a fully normalized 3NF relational database architecture for commercial banking operations. By leveraging InnoDB row-level locking, ACID transactions, foreign-key cascades, composite primary keys on weak entities, and PL/SQL programmability, the database ensures complete reliability, auditability, and protection against data corruption.

7.2 Future Enhancements

1. **Automated Interest Compounding Daemon:** Scheduling the cursor-based `sp_batch_credit_savings_interest` routine via MySQL Event Scheduler to run automatically on the last calendar day of each quarter.
2. **Two-Phase Commit (2PC) Protocol:** Expanding atomic transactions across distributed database shards using XA transactions (`XA START` , `XA PREPARE` , `XA COMMIT`).
3. **Partitioning High-Volume Ledgers:** Range-partitioning the `Transactions` table by `Transaction_date` (quarterly intervals) to maintain microsecond query latencies over multi-million row archives.

---

8. References

1. Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems* (7th ed.). Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts* (7th ed.). McGraw-Hill.
3. MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/
4. Date, C. J. *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
5. Course Laboratory Manual: CSC2131 — Relational Database Management Systems Lab, Department of Computer Science and Engineering, Manipal University Jaipur (2026–2027).

9. Appendix A — SQL Script and Relational Algebra

9.1 Relational Schema Notation

- **Customers** (customer_id, First_name, Last_name, email, phone_number, dob, address, kyc_status, password_hash)
- **Employee** (employee_id, First_name, Last_name, email, role, branch_name, salary, password_hash)
- **Accounts** (Account_id, Account_number, ifsc_code, branch_name, account_type, balance, min_balance, opened_date, status, customer_id (FK))
- **Cards** (card_id, Account_id (FK), Card_number, card_type, cvv, pin_hash, daily_limit, card_status, expiry_date)
- **Transactions** (transaction_id, amount, transaction_type, description, Transaction_date, account_id (FK), processed_by (FK))
- **Loan** (loan_id, loan_type, Attribute, amount, interest_rate, tenure, start_date, status, customer_id (FK), approved_by (FK))
- **Loan_payments** (payment_id, loan_id (FK), amount, payment_date, status, account_id (FK))
- **Account_Tiers** (Tier_Name, Min_Balance, Max_Balance, Benefits)
- **employee_audit** (audit_id, employee_id, action, details, timestamp)

9.2 Complete MySQL DDL: Create Database and Tables

```
CREATE DATABASE IF NOT EXISTS BankDB;
USE BankDB;

-- 1. Customers Table
CREATE TABLE IF NOT EXISTS Customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    First_name VARCHAR(50) NOT NULL,
    Last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    dob DATE NOT NULL,
    address VARCHAR(150) DEFAULT 'Jaipur, Rajasthan',
    kyc_status VARCHAR(20) NOT NULL DEFAULT 'Verified',
    password_hash VARCHAR(255) NOT NULL DEFAULT 'cust123',
    CONSTRAINT unq_cust_email UNIQUE (email),
    CONSTRAINT unq_cust_phone UNIQUE (phone_number)
) ENGINE=InnoDB;

-- 2. Employee Table
CREATE TABLE IF NOT EXISTS Employee (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    First_name VARCHAR(50) NOT NULL,
    Last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    role VARCHAR(30) NOT NULL DEFAULT 'Loan Officer',
    branch_name VARCHAR(50) NOT NULL DEFAULT 'MUJ Main Branch',
    salary DECIMAL(10, 2) NOT NULL DEFAULT 60000.00,
    password_hash VARCHAR(255) NOT NULL DEFAULT 'emp123',
    CONSTRAINT chk_emp_salary CHECK (salary > 0)
) ENGINE=InnoDB;

-- 3. Accounts Table
CREATE TABLE IF NOT EXISTS Accounts (
    Account_id INT AUTO_INCREMENT PRIMARY KEY,
    Account_number VARCHAR(20) NOT NULL,
    ifsc_code VARCHAR(11) NOT NULL DEFAULT 'BANK0001001',
    branch_name VARCHAR(50) NOT NULL DEFAULT 'Jaipur Main',
    account_type VARCHAR(20) NOT NULL DEFAULT 'Savings',
    balance DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    min_balance DECIMAL(12, 2) NOT NULL DEFAULT 1000.00,
    opened_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    status VARCHAR(20) NOT NULL DEFAULT 'Active',
    customer_id INT NOT NULL,
    CONSTRAINT unq_acc_num UNIQUE (Account_number),
    CONSTRAINT chk_acc_balance CHECK (balance >= 0),
    CONSTRAINT fk_accounts_customer FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 4. Cards Table (Weak Entity)
CREATE TABLE IF NOT EXISTS Cards (
    card_id INT AUTO_INCREMENT,
    Card_number VARCHAR(16) NOT NULL,
    card_type VARCHAR(20) NOT NULL DEFAULT 'Debit',
    cvv VARCHAR(3) NOT NULL DEFAULT '101',
    pin_hash VARCHAR(255) NOT NULL DEFAULT '1234',
    daily_limit DECIMAL(10, 2) NOT NULL DEFAULT 50000.00,
    card_status VARCHAR(20) NOT NULL DEFAULT 'Active',
    expiry_date DATE NOT NULL,
    Account_id INT NOT NULL,
    PRIMARY KEY (card_id, Account_id),
    CONSTRAINT unq_card_num UNIQUE (Card_number),
    CONSTRAINT chk_card_limit CHECK (daily_limit > 0),
    CONSTRAINT fk_cards_account FOREIGN KEY (Account_id)
        REFERENCES Accounts(Account_id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 5. Transactions Table
CREATE TABLE IF NOT EXISTS Transactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    amount DECIMAL(12, 2) NOT NULL,
    transaction_type VARCHAR(30) NOT NULL DEFAULT 'Deposit',
```

```
    description VARCHAR(150) DEFAULT 'Counter Transaction',
    Transaction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    account_id INT NOT NULL,
    processed_by INT NULL,
    CONSTRAINT chk_trans_amount CHECK (amount > 0),
    CONSTRAINT fk_trans_account FOREIGN KEY (account_id)
        REFERENCES Accounts(Account_id) ON DELETE CASCADE,
    CONSTRAINT fk_trans_employee FOREIGN KEY (processed_by)
        REFERENCES Employee(employee_id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- 6. Loan Table
CREATE TABLE IF NOT EXISTS Loan (
    loan_id INT AUTO_INCREMENT PRIMARY KEY,
    loan_type VARCHAR(30) NOT NULL,
    Attribute VARCHAR(50) NOT NULL DEFAULT 'Secured',
    amount DECIMAL(12, 2) NOT NULL,
    interest_rate DECIMAL(5, 2) NOT NULL,
    tenure INT NOT NULL,
    start_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    status VARCHAR(20) NOT NULL DEFAULT 'Pending',
    customer_id INT NOT NULL,
    approved_by INT NULL,
    CONSTRAINT chk_loan_amount CHECK (amount > 0),
    CONSTRAINT chk_loan_interest CHECK (interest_rate >= 0),
    CONSTRAINT chk_loan_tenure CHECK (tenure > 0),
    CONSTRAINT fk_loan_customer FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id) ON DELETE CASCADE,
    CONSTRAINT fk_loan_employee FOREIGN KEY (approved_by)
        REFERENCES Employee(employee_id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- 7. Loan_payments Table (Weak Entity)
CREATE TABLE IF NOT EXISTS Loan_payments (
    payment_id INT AUTO_INCREMENT,
    amount DECIMAL(12, 2) NOT NULL,
    payment_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    status VARCHAR(20) NOT NULL DEFAULT 'Paid',
    loan_id INT NOT NULL,
    account_id INT NULL,
    PRIMARY KEY (payment_id, loan_id),
    CONSTRAINT chk_payment_amount CHECK (amount > 0),
    CONSTRAINT fk_pay_loan FOREIGN KEY (loan_id)
        REFERENCES Loan(loan_id) ON DELETE CASCADE,
    CONSTRAINT fk_pay_account FOREIGN KEY (account_id)
        REFERENCES Accounts(Account_id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- 8. Employee Audit Trail Table
CREATE TABLE IF NOT EXISTS employee_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT NOT NULL,
    action VARCHAR(10) NOT NULL,
    details VARCHAR(150) DEFAULT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

9.3 Sample Data: Realistic INSERT Statements

```
-- Customers
INSERT INTO Customers (customer_id, First_name, Last_name, email, phone_number, dob, address, kyc_status) VALUES
(1, 'Aarav', 'Sharma', 'aarav.sharma@gmail.com', '9876543210', '1995-04-12', 'Jaipur, Rajasthan', 'Verified'),
(2, 'Priya', 'Verma', 'priya.verma@yahoo.com', '9823456781', '1998-09-23', 'Jaipur, Rajasthan', 'Verified'),
(3, 'Rohan', 'Mehta', 'rohan.mehta@gmail.com', '9712345678', '1990-11-05', 'Mansarovar, Jaipur', 'Verified'),
(4, 'Sneha', 'Patel', 'sneha.patel@outlook.com', '9632587410', '1997-02-18', 'Vaishali, Jaipur', 'Verified'),
(5, 'Vikram', 'Singh', 'vikram.singh@gmail.com', '9512348765', '1988-07-30', 'Malviya Nagar, Jaipur', 'Pending');

-- Employee
INSERT INTO Employee (employee_id, First_name, Last_name, email, role, branch_name, salary) VALUES
(1, 'Rajesh', 'Khanna', 'rajesh.khanna@bankdb.com', 'Branch Manager', 'MUJ Main Branch', 85000.00),
(2, 'Karan', 'Mehta', 'karan.mehta@bankdb.com', 'Senior Teller', 'MUJ Main Branch', 52000.00),
(3, 'Ananya', 'Roy', 'ananya.roy@bankdb.com', 'Loan Underwriter', 'MUJ Main Branch', 58000.00);

-- Accounts
INSERT INTO Accounts
(Account_id, Account_number, ifsc_code, branch_name, account_type, balance, min_balance, opened_date, status, customer_id)
VALUES
(101, 'ACC10001', 'BANK0001001', 'Jaipur Main', 'Savings', 61923.15, 1000.00, '2023-01-15', 'Active', 1),
(102, 'ACC10002', 'BANK0001001', 'Jaipur Main', 'Current', 114901.00, 5000.00, '2022-06-20', 'Active', 2),
(103, 'ACC10003', 'BANK0001001', 'Jaipur Main', 'Savings', 45000.00, 1000.00, '2023-03-10', 'Active', 3),
(104, 'ACC10004', 'BANK0001001', 'Jaipur Main', 'Salary', 89500.00, 0.00, '2021-11-05', 'Active', 4),
(105, 'ACC10005', 'BANK0001001', 'Jaipur Main', 'Savings', 12000.00, 1000.00, '2024-02-01', 'Active', 5);

-- Cards (Weak Entity)
INSERT INTO Cards (card_id, Account_id, Card_number, card_type, cvv, daily_limit, card_status, expiry_date) VALUES
(1, 101, '4532789012345678', 'Debit', '101', 50000.00, 'Active', '2029-12-31'),
(2, 102, '5425234567890123', 'Credit', '202', 150000.00, 'Active', '2028-06-30'),
(3, 103, '4716987654321098', 'Debit', '303', 25000.00, 'Frozen', '2030-01-31');

-- Transactions
INSERT INTO Transactions (transaction_id, amount, transaction_type, description, Transaction_date, account_id,
processed_by) VALUES
(301, 15000.00, 'Deposit', 'Branch Counter Cash Deposit', '2026-10-20 10:15:00', 101, 2),
(302, 5000.00, 'Withdrawal', 'ATM Cash Dispense', '2026-10-20 11:30:00', 101, NULL),
(303, 10000.00, 'Transfer Out', 'Transfer to Acc #101', '2026-10-21 14:00:00', 102, 1),
(304, 10000.00, 'Transfer In', 'Transfer from Acc #102', '2026-10-21 14:00:00', 101, 1);

-- Loan
INSERT INTO Loan
(loan_id, loan_type, Attribute, amount, interest_rate, tenure, start_date, status, customer_id, approved_by)
VALUES
(201, 'Home Loan', 'Secured', 2500000.00, 8.50, 240, '2024-01-10', 'Approved', 1, 1),
(202, 'Personal Loan', 'Unsecured', 300000.00, 12.00, 36, '2024-05-15', 'Approved', 2, 1),
(203, 'Auto Loan', 'Secured', 800000.00, 9.25, 60, '2024-08-01', 'Pending', 3, NULL);

-- Loan Payments (Weak Entity)
INSERT INTO Loan_payments (payment_id, loan_id, amount, payment_date, status, account_id) VALUES
(1, 201, 21696.52, '2026-08-01', 'Paid', 101),
(2, 201, 21696.52, '2026-09-01', 'Paid', 101),
(3, 202, 9964.29, '2026-08-15', 'Paid', 102);
```

9.4 20 Representative SQL Queries

```
-- 1. Display all active accounts
SELECT * FROM Accounts WHERE status = 'Active';

-- 2. Select specific customer contact details
SELECT customer_id, First_name, Last_name, phone_number, email FROM Customers;

-- 3. WHERE filter with minimum balance limit
SELECT Account_number, balance FROM Accounts WHERE balance >= 50000.00;

-- 4. LIKE: Find customers with Gmail address
SELECT First_name, Last_name, email FROM Customers WHERE email LIKE '%@gmail.com';

-- 5. DISTINCT account types
SELECT DISTINCT account_type FROM Accounts;

-- 6. BETWEEN: Transactions between ₹5,000 and ₹50,000
SELECT transaction_id, amount, transaction_type FROM Transactions WHERE amount BETWEEN 5000.00 AND 50000.00;

-- 7. IN: Accounts of specific branch locations
SELECT Account_number, branch_name, balance FROM Accounts WHERE branch_name IN ('Jaipur Main', 'Mansarovar');

-- 8. ORDER BY balance descending
SELECT Account_number, account_type, balance FROM Accounts ORDER BY balance DESC;

-- 9. Multi-table Join: Accounts with Customer Details
SELECT A.Account_number, A.account_type, A.balance, C.First_name, C.Last_name
FROM Accounts A
INNER JOIN Customers C ON A.customer_id = C.customer_id;

-- 10. Multi-table Join: Transactions with Account and Customer Name
SELECT T.transaction_id, T.amount, T.transaction_type, A.Account_number, C.First_name
FROM Transactions T
INNER JOIN Accounts A ON T.account_id = A.Account_id
INNER JOIN Customers C ON A.customer_id = C.customer_id;

-- 11. LEFT JOIN: All customers and their loans (including customers with no loans)
SELECT C.First_name, C.Last_name, L.loan_type, L.amount
FROM Customers C
LEFT JOIN Loan L ON C.customer_id = L.customer_id;

-- 12. Aggregate: Total bank deposits and customer count
SELECT COUNT(*) AS total_accounts, SUM(balance) AS total_vault_deposits, AVG(balance) AS avg_balance FROM Accounts;

-- 13. GROUP BY: Account count and total balance by account type
SELECT account_type, COUNT(*) AS count_type, SUM(balance) AS total_balance
FROM Accounts
GROUP BY account_type;

-- 14. GROUP BY with HAVING: High liquidity categories
SELECT account_type, SUM(balance) AS sum_bal
FROM Accounts
GROUP BY account_type
HAVING SUM(balance) > 50000.00;

-- 15. Single-Row Subquery: Accounts with balance above average
SELECT Account_number, balance
FROM Accounts
WHERE balance > (SELECT AVG(balance) FROM Accounts);

-- 16. Correlated Subquery with NOT EXISTS: Customers with no loans
SELECT customer_id, First_name, Last_name
FROM Customers C
WHERE NOT EXISTS (SELECT 1 FROM Loan L WHERE L.customer_id = C.customer_id);

-- 17. UNION ALL: Consolidated directory of Bank Customers and Staff
SELECT customer_id AS ID, CONCAT(First_name, ' ', Last_name) AS Name, 'Customer' AS Role FROM Customers
UNION ALL
SELECT employee_id AS ID, CONCAT(First_name, ' ', Last_name) AS Name, role AS Role FROM Employee;

-- 18. Weak Entity Query: Retrieve active cards for Account #101
```

```sql
SELECT C.Card_number, C.card_type, C.daily_limit, A.Account_number
FROM Cards C
INNER JOIN Accounts A ON C.Account_id = A.Account_id
WHERE A.Account_id = 101 AND C.card_status = 'Active';

-- 19. Amortization Sum: Total loan payments collected per loan
SELECT loan_id, SUM(amount) AS total_paid_to_date
FROM Loan_payments
GROUP BY loan_id;

-- 20. TCL Transaction Simulator with Savepoint & Rollback
START TRANSACTION;
SELECT balance FROM Accounts WHERE Account_id = 102 FOR UPDATE;
SAVEPOINT before_transfer;
UPDATE Accounts SET balance = balance - 5000.00 WHERE Account_id = 102;
ROLLBACK TO SAVEPOINT before_transfer;
COMMIT;
```

9.5 Formal Relational Algebra Expressions

Notation Reference:

- **σ** : Selection (filters rows based on predicate)
- **π** : Projection (extracts specified attributes)
- ⋈ : Natural / Theta Join
- × : Cartesian Product
- ∪ : Union
- − : Set Difference

1. All Active Accounts:

$\sigma_{status = 'Active'}(Accounts)$

2. Customer Names and Email Addresses:

$\pi_{First_name, Last_name, email}(Customers)$

3. Customers with Balance Exceeding ₹50,000:

$\pi_{First_name,\ Last_name,\ balance}(Customers \bowtie_{Customers.customer_id\ =\ Accounts.customer_id} (\sigma_{balance\ >\ 50000}(Accounts)))$

4. Active Debit Cards Linked to Account #101:

$\pi_{Card_number,\ daily_limit}(\sigma_{Account_id\ =\ 101\ \wedge\ card_type\ =\ 'Debit'\ \wedge\ card_status\ =\ 'Active'}(Cards))$

5. Customers Who Have Never Applied for a Loan:

$\pi_{customer_id}(Customers) - \pi_{customer_id}(Loan)$

6. Union of Customer and Employee Email Addresses:

$\pi_{email}(Customers) \cup \pi_{email}(Employee)$

7. Approved Loans with Principal Exceeding ₹5,00,000:

$\pi_{loan_id,\ amount,\ interest_rate}(\sigma_{status\ =\ 'Approved'\ \wedge\ amount\ >\ 500000}(Loan))$

9.6 Extended Relational Algebra (Aggregation γ)

Classical relational algebra does not support mathematical aggregations. The extended relational algebra operator **γ** defines grouping and aggregation:

1. Accounts Grouped by Type with Total Balance and Count:

$\gamma_{account_type;\ COUNT(Account_id) \rightarrow TotalAccounts,\ SUM(balance) \rightarrow TotalBalance}(Accounts)$

2. Approved Loans Grouped by Type with Average Interest Rate and Total Disbursed:

$\gamma_{loan_type;\ AVG(interest_rate) \rightarrow AvgRate,\ SUM(amount) \rightarrow TotalDisbursed}(\sigma_{status\ =\ 'Approved'}(Loan))$

9.7 Database Verification Scripts

Run these commands in MySQL Workbench to verify database initialization:

```
SHOW TABLES;
DESCRIBE Customers;
DESCRIBE Accounts;
DESCRIBE Cards;
DESCRIBE Transactions;
DESCRIBE Loan;
DESCRIBE Loan_payments;

SELECT COUNT(*) AS TotalCustomers FROM Customers;
SELECT COUNT(*) AS TotalEmployees FROM Employee;
SELECT COUNT(*) AS TotalAccounts FROM Accounts;
SELECT COUNT(*) AS TotalCards FROM Cards;
SELECT COUNT(*) AS TotalTransactions FROM Transactions;
SELECT COUNT(*) AS TotalLoans FROM Loan;
SELECT COUNT(*) AS TotalLoanPayments FROM Loan_payments;
```

9.8 Execution & Deployment Notes

1. **Database Name:** The production schema is named `BankDB`. Ensure `.env` has `DB_NAME=BankDB`.
2. **InnoDB Storage Engine:** All tables must explicitly declare `ENGINE=InnoDB` to ensure foreign key cascades and transactional savepoint rollbacks execute properly.
3. **Safe Updates Mode:** When running ad-hoc `UPDATE` queries in MySQL Workbench, disable safe mode (`SET SQL_SAFE_UPDATES = 0;`) if targeting non-primary key columns.
4. **Referential Integrity Ordering:** Always execute `INSERT` statements into parent tables (`Customers`, `Employee`) before dependent child tables (`Accounts`, `Loan`), and child tables before weak entity tables (`Cards`, `Loan_payments`).